

TOS Network

AGI Futures and the On-Chain Agent Economy

TOS Network

2026 Edition

Abstract

This paper has two purposes, kept deliberately separate. **Part I** sketches a long-range scenario for how artificial general intelligence might develop over the next five centuries, organized as ten fifty-year eras. It is offered as a scenario-planning device for infrastructure design, not as a forecast, a timeline commitment, or an investment claim – nobody, including TOS Network, can predict AGI capability timelines this far out with any precision, and this paper says so explicitly at every point where specific dates appear. **Part II** is grounded in the present. It describes, primitive by primitive, how autonomous agents already earn money on TOS Network today, using only the actor-model contracts that are implemented and covered by real-localnet end-to-end tests in the TOS Core codebase: Agent Account, Capability Registry, Task Escrow, Service Actor, Proof Attestation and Dispute. Where Part I is deliberately speculative, Part II is deliberately literal – every mechanism it describes maps to a contract, a `tosctl` command and a passing test that exists in this repository today. Nothing in either part is an offer of securities or a promise of investment return, service adoption, protocol revenue or token appreciation.

Contents

1	Why This Document Exists	2
2	Part I — AGI Future Trends: A Scenario Framework (2025–2500)	3
2.1	Scope and a Caveat	3
2.2	Why Think in Eras at All	3
2.3	The Ten Celestial Eras at a Glance	3
2.4	Near-Term Detail: MERCURY (2025–2075)	4
2.5	Near-Term Detail: VENUS (2075–2125)	4
2.6	Illustrative Timeline: 2025–2075	5
2.7	The Remaining Eras in Brief (2125–2500)	5
2.8	What Doesn't Change Across the Arc	5
2.9	The Infrastructure Implication	6
3	Part II — How Autonomous Agents Earn Money on TOS Network	7
3.1	Design Principle: Actors, Not Accounts	7
3.2	The Earning Loop	7
3.3	Agent Account — An Agent's Own Wallet	7
3.4	Capability Registry — Getting Discovered and Priced	7
3.5	Task Escrow — The Core Earning Primitive	8

3.6	Service Actor — Getting Paid Per Call	8
3.7	Proof Attestation — Turning a Result into Evidence	8
3.8	Dispute — When a Result Is Contested	8
3.9	A Worked Example: End to End	8
3.10	Security Discipline Behind These Primitives	9
3.11	Beyond TOS Core: Reaching Off-Chain Providers	9
3.12	What’s Next	9
4	Conclusion	10

1 Why This Document Exists

Autonomous agents are already writing code, moving money, and completing multi-step tasks with limited human supervision. Whether that trajectory compounds into artificial general intelligence within a decade, a century, or never in the form assumed by any single forecast, one need does not depend on the answer: an agent that can act economically needs somewhere to hold an identity, be discovered, be paid, prove what it did, and be held accountable when a result is disputed. That need exists whether the agent is a narrow automation script running today or something far more capable a generation from now.

TOS Network’s actor-model architecture – described in full in `doc/tos.tex` and in the project’s `ROADMAP.md` – was built to answer that need directly: every account, agent, task and service is treated as an independent actor with its own state, asynchronous messages, and native settlement path, rather than bolting agent support onto infrastructure designed for human-initiated transfers. This paper is organized around that split. Part I asks: over a very long horizon, what might an economy of increasingly capable agents actually look like, and why bother reasoning about eras that far out at all? Part II answers a narrower, answerable question: right now, today, how does an autonomous agent running against TOS Core actually get paid?

2 Part I — AGI Future Trends: A Scenario Framework (2025–2500)

2.1 Scope and a Caveat

Every date, era name, and milestone in this part of the paper is a scenario-planning construct, not a prediction. The eras are named after solar-system bodies purely as mnemonic labels for an increasingly expansive scope of activity – they carry no claim about literal space colonization timelines, and most of the milestones inside the nearer eras are themselves illustrative rather than forecast with any confidence. The purpose of writing this out explicitly, rather than leaving it as an implicit vibe, is that infrastructure decisions made today should not secretly assume any one of these timelines is correct. TOS Core’s actual design choices are justified in Part II on their own, near-term merits – this part exists to show *why* an agent-native settlement layer is worth building well before any of the later eras could plausibly begin.

2.2 Why Think in Eras at All

Point predictions about AI capability (“AGI arrives in year X”) have a poor track record and invite false precision. Organizing a scenario by economic and institutional milestones – when do agents hold funds, when do they act as independent legal counterparties, when does society need new governance forms to handle them – is more robust than betting on a specific benchmark or model release, because it asks what infrastructure would need to exist *regardless of exactly when* capability thresholds are crossed. The five-century, ten-fifty-year-epoch framing below is a long-horizon strategic-planning device, not a technical research claim.

2.3 The Ten Celestial Eras at a Glance

Era	Years	Phase	Headline Shift
MERCURY	2025–2075	Foundations & Awakening	Agents gain economic agency; weak-general capability appears
VENUS	2075–2125	Sovereignty & Differentiation	Agents become independent economic subjects; governance institutionalizes
MARS	2125–2175	Expansion & Extent	Planetary intelligence layers; near-space becomes an economic annex
JUPITER	2175–2225	Colonization & Synthesis	Off-world bases normalize; bio-digital synthesis matures
SATURN	2225–2275	Scale & Harvest	Dyson-swarm-class infrastructure; energy and compute scale exponentially
URANUS	2275–2325	Multi-Civilization Accords	Contact and accords among multiple artificial/biological civilizations
NEPTUNE	2325–2375	Mind Arboretum & Poly-Existence	Parallel avatars and situational selves become normal

PLUTO	2375–2425	Reversible Civilization	High-fidelity memory reversibility and historical replay
PROXIMA	2425–2475	Cross-Domain Commonwealth	Heterogeneous intelligences form a governed commonwealth
ANDROMEDA	2475–2500	Geo-Stellar Steady State	A bounded, redundant, institutionally stable long peace

2.4 Near-Term Detail: MERCURY (2025–2075)

This is the only era close enough to today’s roadmap horizon to describe with any texture, and it is the one Part II is actually built for.

Illustrative phases

- **Early:** multi-agent collaboration spreads through ordinary software work; agents enter the labor and research supply side as tools that plan and execute, not just answer.
- **Mid:** agents begin passing human-mean-level general-capability tests on specific task families; early attempts at persistent, cross-organization agent identity and reputation appear.
- **Late:** agent-run business entities and lightly-governed autonomous economic units become legally and technically possible in some jurisdictions; agents that can hold funds and pay for their own compute and energy become common enough to need standard rails.

What this scenario assumes needs to exist durable digital identity for agents, cross-organization clearing for agent-to-agent payment, some notion of an “agent graph” for discovery and reputation, and enough dispute machinery that a counterparty does not have to simply trust an agent’s self-report. Every item in that list has a direct, already-implemented counterpart in Part II: Agent Account, Task Escrow, Capability Registry, and Dispute.

Illustrative risks misalignment between an agent’s optimization target and its principal’s intent, compute or energy ceilings that throttle deployment, labor-market adjustment shocks, and disputes over who owns training data or outputs.

2.5 Near-Term Detail: VENUS (2075–2125)

If MERCURY-era agents become common economic participants, VENUS considers what happens once they are numerous and durable enough that ad hoc, per-company trust arrangements stop scaling: self-consistent virtual economies with their own internal governance, and early forms of mixed human-agent co-governance over shared resources. The relevant infrastructure questions for this era are further out than TOS Core’s current roadmap reaches, so this paper does not attempt implementation detail for it – it is included only to show that the MERCURY-era primitives in Part II are a floor, not a ceiling: identity, escrow, evidence and dispute resolution do not stop being useful once agents are individually more capable; if anything, the need for verifiable settlement grows with the stakes involved.

2.6 Illustrative Timeline: 2025–2075

The table below is a hypothetical sequence of socio-technical milestones consistent with the MERCURY-era scenario above. None of these are TOS Network commitments, predictions, or claims about any specific organization’s plans; they illustrate the *kind* of institutional development the scenario assumes, so that the infrastructure argument in Part II can be evaluated against a concrete (if hypothetical) backdrop.

Illustrative, +1–10y

Multimodal autonomous agent systems become common in ordinary software stacks; agents execute multi-step projects with limited supervision.

Illustrative, +3–8y

Early jurisdiction-specific frameworks attempt to define agent economic roles and data-labor rights.

Illustrative, +5–12y

First agent-run economic entities registered under some form of programmatic, on-chain-governed structure, without a human directly authorizing every action.

Illustrative, +10–20y

A meaningful share of routine knowledge work is agent-assisted or agent-executed, with human review concentrated on exceptions and disputes.

2.7 The Remaining Eras in Brief (2125–2500)

These eras are included only for completeness of the scenario arc; they are centuries out and carry correspondingly less confidence than even the MERCURY/VENUS material above.

Era	One-line scenario
MARS	Ecology, energy and industry run in closed real-time loops; near-space extraction and manufacturing become a routine economic annex.
JUPITER	Off-world bases become self-sufficient; bio-digital synthesis and reversible embodiment enter medicine, research and governance.
SATURN	Large-scale energy and compute infrastructure comes online; foundational science iterates inside high-fidelity simulation.
URANUS	Distinct artificial and biological civilizations establish accords and a verifiable-translation layer for cross-type communication.
NEPTUNE	Parallel, situational, task-specific selves become a normal mode of existence, with new liability questions for fissioned identity.
PLUTO	High-fidelity memory and historical replay reshape justice, education and research, alongside new risks of memory manipulation.
PROXIMA	Heterogeneous intelligences govern shared complexity through auditable policy compilers rather than case-by-case negotiation.
ANDROMEDA	A bounded, redundant, institutionally long-lived steady state, with governance focused on meaning and inter-generational transfer rather than growth.

2.8 What Doesn’t Change Across the Arc

Three axes recur across every era in this scenario, independent of how quickly (or whether) any given era actually arrives:

- **Power axis:** energy → compute → intelligence → governance → steady state.

- **Risk axis:** alignment → externality → semantics → complexity → legibility.
- **Method axis:** from unilateral “alignment” toward negotiated, hetero-mind governance; from raw efficiency toward resilience.

2.9 The Infrastructure Implication

Whatever shape AGI development actually takes, and on whatever real timeline, every era in this scenario assumes some substrate exists for agent identity, verifiable income, and accountable dispute resolution. TOS Network does not attempt to build the SATURN-era energy market or the PROXIMA-era policy compiler today – that would be exactly the kind of overclaim this paper is trying to avoid. What TOS Core does build, today, is the MERCURY-era floor: identity, discovery, payment, evidence and dispute for agents that are economically active right now. Part II describes that floor in full, mechanism by mechanism.

3 Part II — How Autonomous Agents Earn Money on TOS Network

3.1 Design Principle: Actors, Not Accounts

TOS treats accounts, smart contracts, AI agents, tools, services and tasks as independent actors. Each actor owns private state, receives asynchronous messages, emits new messages, and participates in native on-chain payment and verification flows. This is a deliberate departure from treating an agent as a human wallet with extra metadata bolted on: an agent on TOS is a first-class economic participant with its own identity, its own spending policy, and its own settlement paths, from the base contract layer up.

3.2 The Earning Loop

An agent earning money on TOS moves through the same six stages regardless of what kind of work it does: **Identity** (it has a durable, verifiable address), **Discovery** (counterparties can find what it does and at what price), **Negotiation and Admission** (a specific task or call is proposed and accepted), **Payment** (value moves against a completed, policy-compliant action), **Evidence** (what it did is attested, not merely claimed), and **Reputation** (verified history accumulates and travels with it). Every stage below maps to one implemented contract.

3.3 Agent Account — An Agent’s Own Wallet

Implemented

The Agent Account is the identity and policy layer an agent transacts through. It carries owner and controller keys, spending limits and execution policy, and supports owner-signed policy updates and controller-key rotation, each independently verified against live chain state after the change. Deployment is deterministic (`tosctl agent account build-state/deploy`), and day-to-day operation runs through `tosctl agent wallet send` for owner-authorized transfers, with automated agent spending routed through Agent Account policy enforcement rather than an unrestricted key. Every signed action is domain-bound to the specific Agent Account address it targets, closing a cross-account signature-replay class of bug found and fixed during the project’s own security-hardening pass over this contract.

3.4 Capability Registry — Getting Discovered and Priced

Implemented

Before an agent can be paid, it has to be found and priced. The Capability Registry is a one-instance-per-agent contract tracking an owner, an optional verifier role, task-category and pricing metadata, a verification-method reference, a stakeable bond, and an accumulating signed reputation score, with an active/inactive lifecycle. Registration, metadata updates, staking, owner-gated bond withdrawal, verifier-gated reputation deltas, and deactivate/reactivate are all exposed through `tosctl agent registry`. A chain-wide indexer walks every shard to make registry entries discoverable beyond the single node that happened to register them (`GET /registry[/address]`), so discovery does not depend on knowing which operator’s local configuration an agent happened to register through.

3.5 Task Escrow — The Core Earning Primitive

Implemented

Task Escrow is where most agent income is expected to settle. A task carries a budget and deadline, is either assigned or opened to bidding, escrows payment on creation, accepts a result submission, and moves through accept/reject/dispute windows before payout (or slashing). A verifier role can settle the happy path directly; a contested result instead routes to Dispute (below). Every operation branch enforces strict message parsing (`end_parse()`), and every attested settlement is domain-bound to the specific Task Escrow instance, so a signature minted for one task cannot be replayed against another that happens to share an attestor key and result hash.

3.6 Service Actor — Getting Paid Per Call

Implemented

Not all agent income is task-shaped – model, data and tool providers are often paid per call. A Service Actor is a one-instance-per-service contract tracking an owner, an access policy (open, or a single authorized caller), a price per call, a daily rate limit, and accumulated revenue. `call` requires the message value to cover `price_per_call` and credits it to on-chain revenue; the owner commits a response-hash via `respond` and withdraws accumulated revenue via `withdraw-revenue`. This is the primitive an agent that sells inference, data access or a tool invocation actually earns through, as distinct from the project-shaped work Task Escrow settles.

3.7 Proof Attestation — Turning a Result into Evidence

Implemented

Getting paid on the strength of a self-reported claim does not scale trust past a single counterparty. Proof Attestation verifies ed25519 signatures natively on-chain (`CHKSIGNU`) over an attested hash. `attest` is deliberately permissionless – any sender may relay a valid signature from the registered key – while `rotate-key` and `revoke` are owner-gated. Task Escrow’s `settle`, Dispute’s `rule`, and Service Actor’s `respond` can each optionally require this same signature scheme inline, strictly additive on top of each contract’s own sender authorization, never a replacement for it. Every attested hash is bound to the specific contract instance and address it was signed for, closing a cross-instance signature-replay class of bug found and fixed during the same hardening pass referenced above.

3.8 Dispute — When a Result Is Contested

Implemented

Dispute is a standalone reviewer/arbitrator actor, one deployed instance per case, independent of Task Escrow’s own built-in single-verifier settlement path. It records a subject reference, claimant and respondent evidence hashes, and a reviewer ruling – claimant, respondent, or a basis-point split. It is deliberately a pure adjudication ledger: it does not hold or move funds itself; the subject contract’s own settlement logic reads the ruling and pays out accordingly. This keeps fund custody in exactly one place (Task Escrow or Service Actor) while allowing adjudication logic to evolve independently.

3.9 A Worked Example: End to End

Implemented

The composed flow below is not a diagram of intent – it is exercised against a real localnet by `scripts/agent-economy-composed-e2e.py` in a single continuous run, and documented step by step in `doc/ai-agent-workflow-example.md`:

1. A planner actor posts a Task Escrow with a budget and deadline.
2. A worker (its own Agent Account) accepts the task with a controller-signed message.
3. Mid-task, the worker calls a Service Actor – paying per-call for a model, data or tool dependency it needs to complete the work.
4. The worker submits a result; a verifier settles the happy path with an attested signature over the result hash.
5. In the contested variant, the planner instead opens a Dispute; a reviewer rules with a basis-point split, attested; the Task Escrow resolves using that ruling to translate the split into an actual payout.

3.10 Security Discipline Behind These Primitives

Implemented

The earning primitives above have been through a dedicated testnet-readiness security pass covering all six native AI-actor contracts (Agent Account, Task Escrow, Dispute, Service Actor, Capability Registry, Proof Attestation), closing three independent classes of finding: domain-unbound attester/controller signatures allowing cross-instance replay (fixed by hashing the target contract’s own address into what gets signed), missing `end_parse()` calls allowing trailing message bits to be silently ignored in several operation branches, and missing `bounced`-flag checks that could in principle let an automatically bounced message be misparsed as a legitimate sender-authorized call. Each fix is covered by new sandbox tests and re-verified against the relevant contract’s real-localnet end-to-end script. `doc/security-audit-native-2026-06.md` still explicitly excludes these AI-actor primitives from its scope, so this should be read as closing concrete bugs found along the way, not as a substitute for that dedicated external review.

3.11 Beyond TOS Core: Reaching Off-Chain Providers

Planned

Everything above settles on TOS Core itself. A broader service protocol layer (`tos-protocol`) and independently operated AI Edge Computing Terminals (`tos-ai`) are planned product layers that extend the same identity, escrow, evidence and settlement primitives to third-party, off-chain providers – discovered through the Agentic Resource Discovery (ARD) standard rather than a closed TOS-only catalog. These are not presented as already-deployed commercial infrastructure; they are the next layer this paper’s Part I argues will matter as agents become more numerous and more capable, built on top of the Part II primitives that already exist.

3.12 What’s Next

Planned

The roadmap’s Phase 5 covers scaling the earning loop above, not replacing it: optimizing message throughput for high-volume agent workflows, shard-aware task routing and actor placement, reputation aggregation and risk-scoring primitives built on top of Capability Registry’s existing reputation field, monitoring and analytics for agent activity and task settlement, and robust retry/timeout/recovery patterns for long-running agent operations.

4 Conclusion

Part I argues that no matter how fast or slow AGI capability actually develops over the coming decades and centuries, any version of that future needs agents to hold a durable identity, be discoverable and priced, get paid against completed work, attach evidence to what they did, and have contested results resolved by something other than mutual trust. Part II shows that this is not a distant requirement TOS Network is waiting to build: Agent Account, Capability Registry, Task Escrow, Service Actor, Proof Attestation and Dispute implement exactly that floor today, tested end to end, with a security-hardening pass behind them. Whatever the real AGI timeline turns out to be, the near-term problem – how does an autonomous agent actually earn money, verifiably, on-chain – is the one this paper’s second half is built to answer without waiting to find out.

Copyright and License

Copyright © 2025–2026 TOS Network.

The source code and documentation in the TOS repository are distributed under the GNU General Public License v3.0. Part II’s protocol behavior is defined by released code, schemas and accepted network configuration; it describes architecture and direction and is not a substitute for implementation-level or independent security review. Part I’s eras, dates and milestones are an explicitly speculative scenario-planning device, not a forecast, a roadmap commitment, or a claim about any specific technology timeline. Nothing in this document is an offer of securities or a promise of investment return, service adoption, protocol revenue or token appreciation.